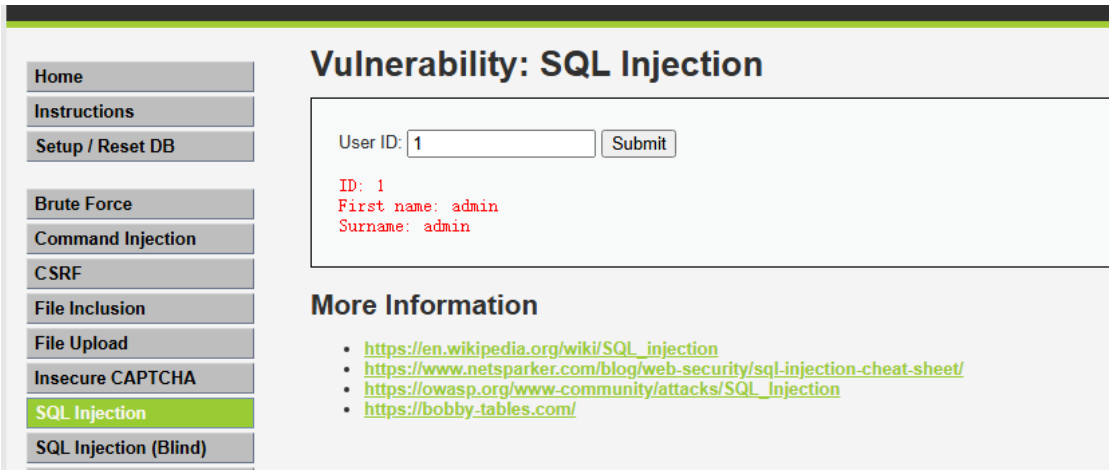
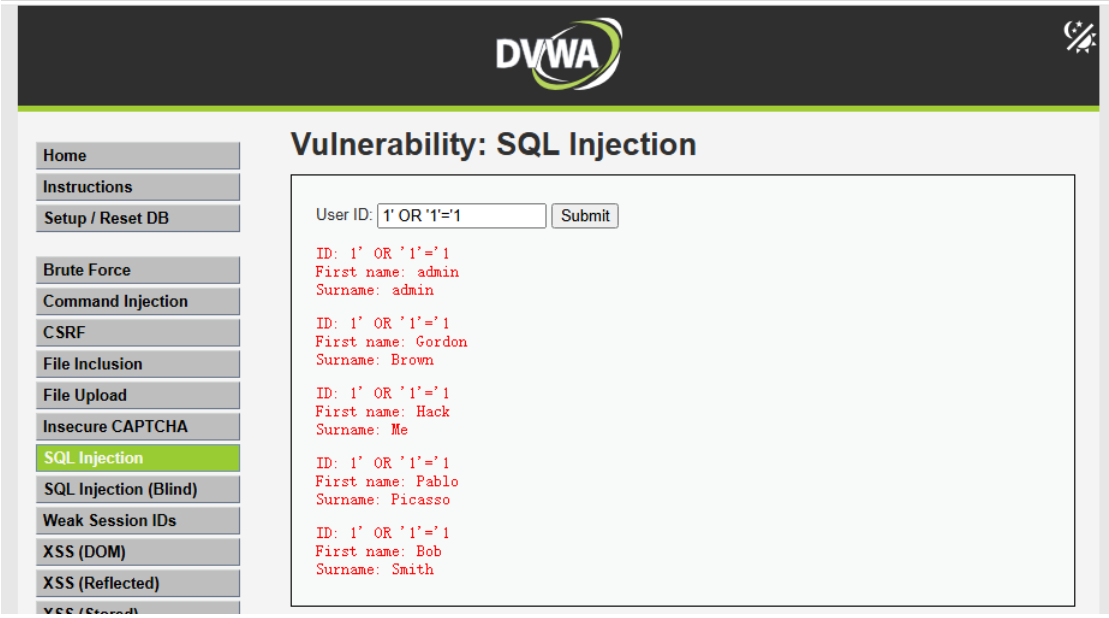
任务 1：数据库与查询条件基础说明

1. 以校园教务系统查询成绩为例，页面展示的成绩、课程、个人信息等动态数据，都统一存储在后台数据库中，网页本身无法储存大量实时业务数据。用户在页面进行输入操作，是为了自定义筛选想要查看的数据，系统需要依靠这份输入内容访问数据库调取对应信息，因此页面输入会直接和数据库查询产生关联。
2. 以教务系统查成绩为例，完整最小业务链路为：输入条件→数据库查询→返回结果。用户输入学号、课程名称作为查询条件，后端携带该条件发起数据库查询操作，数据库检索匹配数据后，将查询结果返回页面进行展示。
3. 用户在网页填写的内容会被后端程序接收，开发人员未做安全处理时，会直接将用户输入的内容拼接组合到 SQL 查询语句中，让用户输入的内容直接成为数据库检索的判断条件，参与数据匹配筛选。
4. 后端无过滤、无转义处理，直接信任并拼接用户输入内容到 SQL 语句中。当用户输入恶意 SQL 语法字符时，恶意内容会被数据库识别并正常执行，篡改原本的查询逻辑，随意查询、泄露甚至修改数据库数据，由此产生 SQL 注入漏洞风险。

任务 2：DVWA SQL Injection Low 现象记录



Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near ''1'' at line 1 in E:\xampp\htdocs\DVWA\vulnerabilities\sqli\source\low.php:11 Stack trace: #0 E:\xampp\htdocs\DVWA\vulnerabilities\sqli\source\low.php(11): mysqli_query(Object(mysqli), 'SELECT first_na...') #1 E:\xampp\htdocs\DVWA\vulnerabilities\sqli\index.php(34): require_once('E:\\xampp\\htdocs...') #2 {main} thrown in **E:\xampp\htdocs\DVWA\vulnerabilities\sqli\source\low.php** on line **11**



1．目标：在授权合法环境中，对 Web 查询交互页面进行输入测试，观察正常输入与异常特殊输入的服务返回区别，分析后端数据处理机制是否存在异常解析问题。

2.对比：普通输入与异常输入对比普通输入为日常合规查询内容，由汉字、数字、普通字母组成，无任何特殊符号与语法字符；异常输入在同一输入位置，添加闭合符号、注释符、逻辑特殊字符等畸形内容，输入长度、提交方式、访问环境完全一致，仅字符构成不同。

3.差异：返回结果差异普通输入提交后，页面正常渲染，精准返回对应查询数据，页面结构完整，无报错、无空白，交互功能正常。异常输入提交后，正常查询数据消失，出现页面空白、接口错误、内容截断或局部功能加载失败，无法完成正常业务查询展示。

4. 为什么这种差异不像“正常查询变化”：

区别于正常查询变化的原因正常查询内容改动，只会改变检索出的内容多少，页面整体结构、运行状态、响应逻辑不会被破坏。本次实验中，仅增加特殊字符就引发程序级报错与页面渲染异常，不是简单检索不到数据，而是后端解析规则被干扰、语句执行出现错误，属于程序处理异常，因此不属于正常查询带来的内容变化。

任务 3：机制分析与差异比较

1. 用户在页面提交的自定义输入，会被后端程序直接获取，在未做安全过滤与转义的情况下，程序将用户输入字符串直接和固定 SQL 语句进行字符串拼接，完整组合成一条新的查询语句。数据库会按照自身语法规则对拼接后的整条语句进行语法解析，识别查询逻辑、条件范围等结构，最后执行这条 SQL 语句，根据执行结果，要么直接回显数据内容，要么产生页面报错、访问异常等行为差异，完整流程为：

输入 — 拼接 — 解析 — 执行 — 回显或行为差异。

2. 特殊字符单独存在时没有任何危害性，只是普通文本符号。SQL 注入发生的核心，是用户可控输入被强行拼入 SQL 查询整体结构，篡改了原有查询的逻辑框架、条件限制与执行范围，破坏了原本预设的查询结构。危险的不是字符本身，而是非法输入改变了数据库解析执行的语句结构，因此根因是查询结构被污染，而非字符自带危险属性。
3. Low 级别完全没有任何防护措施，不对用户输入做限制、过滤与转义，直接原生拼接输入内容到 SQL 语句，特殊符号可以完整带入查询结构，注入构造简单，可直接利用单引号等字符破坏语句结构。Medium 级别增加了基础防护，会对部分特殊危险字符进行简单替换或转义，过滤了常用注入符号，简单的注入载荷会被拦截，无法直接篡改查询结构，提升了注入的难度，但防护手段简陋，仍存在绕过风险，无法彻底杜绝 SQL 注入。
4. 普通 SQL 注入存在数据回显点，注入攻击后，数据库查询的敏感数据、报错信息会直接展示在页面上，攻击者可以直观获取查询结果，依靠页面明文数据完成攻击。盲注也就是盲 SQL 注入，页面没有任何数据回显与详细报错信息，无法直接看到数据库返回内容，攻击者无法直观拿到数据，只能依靠页面响应状态、加载时长、页面内容差异等间接行为变化，逐位猜测、推导数据库数据。
5. 二者根本成因完全一致，都是后端直接拼接用户可控输入，导致 SQL 查询结构被恶意篡改，都存在输入未隔离、语句拼接不安全的核心漏洞。区别在于前端页面反馈机制不同，普通注入有数据回显，以明文数据、报错内容作为攻击证据；盲注屏蔽了直接数据输出，只能以页面响应差异、时间延迟等隐性行为作为判断证据，因此根因一致，只是对外呈现的证据形式不同。

强制判断题：

1. 选 B：因为少量特殊字符不具备攻击能力，真正引发漏洞的是用户输入被随意拼入 SQL 查询结构，篡改了原有查询逻辑，并非字符本身存在危害。
2. 选 B：因为两种注入的底层漏洞、形成根因完全相同，都是查询结构拼接污染，核心区别在于页面输出反馈不同，一个直接回显数据，一个无直观数据，仅靠行为差异判断，也就是证据形式不同。

任务 4：SQL 注入防护清单与解释

1.参数化查询：通过预编译语句将 SQL 逻辑与用户输入分离，彻底阻断恶意 SQL 代码与原有语句拼接执行的路径，从源头杜绝 SQL 注入漏洞，属于根因控制。它不能替代输入校验、错误处理和最小权限，仅解决拼接问题，无法防范非法业务数据、敏感信息泄露等其他风险。

2. 输入校验：在数据进入应用前，对用户输入进行规则校验，过滤包含 SQL 关键字、特殊符号的恶意内容，拦截明显攻击请求，属于暴露面控制。它不能替代参数化查询，无法防御编码绕过、字符变形等复杂注入攻击，仅能减少攻击入口。

3. 错误处理：屏蔽数据库原生报错和程序详细异常信息，不向攻击者返回数据库版本、表结构等敏感线索，避免攻击者精准构造注入语句，属于暴露面控制。它不能替代参数化查询和输入校验，仅能隐藏信息，无法阻止已发生的注入攻击，也不能修复漏洞本身。

4. 最小权限：为应用连接数据库的账号分配最低必要权限，禁止删除、修改表结构等高危操作，即使发生 SQL 注入，也能限制攻击者的破坏范围，属于后果范围控制。它不能替代参数化查询，无法防止注入漏洞产生，仅能降低攻击造成的损失。

5. 补充措施（ORM 框架）：通过框架自动处理 SQL 编写，避免手动拼接语句，降低漏洞产生概率，属于根因辅助控制。它不能替代参数化查询，复杂 SQL 场景下仍需规范编写，无法完全规避注入风险

收口项 1：SQL 注入因果链/防护关系图

流程环节	**环节核心内容**	**因果作用**	**根因 / 风险定位**	**对应防护控制点**
输入点	用户可控外部输入，存在特殊符号、SQL 恶意关键字等恶意内容	攻击起始入口，提供恶意载荷传入通道	暴露面风险，外部输入不可控	输入校验：过滤非法字符与注入特征，前置拦截攻击载荷
查询构造	程序采用字符串拼接方式组合 SQL 语句与用户输入	篡改原有 SQL 逻辑，形成注入漏洞核心条件	**核心根因**：输入与 SQL 语句未隔离	参数化查询：分离 SQL 结构与用户输入，根治拼接漏洞
数据库解析 / 执行	数据库解析拼接后的恶意 SQL，执行非法查询、篡改、删除等操作	攻击正式生效，触发违规数据库行为	执行层风险，缺少权限约束	最小权限策略：限制数据库账号操作范围，约束执行危害
结果差异或行为证据	页面报错、数据泄露、内容异常、访问越权等可观测现象	暴露攻击痕迹，为漏洞探测与利用提供线索	信息暴露风险，敏感内容外溢	统一错误处理：屏蔽数据库原生报错，隐藏攻击行为证据

流程环节	环节核心内容	因果作用	根因 / 风险定位	对应防护控制点
根因判断	根本原因是动态 SQL 拼接、输入无隔离；辅助诱因是权限过大、信息泄露	串联全链路风险，明确漏洞本质	注入漏洞核心判定依据	多层防护联动：根因 + 暴露面 + 后果全维度防护管控

收口项 2：学生自评表

1.我本章最清楚的一点是：能够清晰区分 SQL 注入的攻击原理、完整因果链路，同时熟练掌握参数化查询、输入校验、最小权限等各类防护措施的作用层级，明确根因控制、暴露面控制、后果范围控制三者的区别与对应场景。

2.我最容易混淆的一点是：容易混淆不同防护措施的适用环节，偶尔会分不清错误处理、WAF 等防护手段具体归属暴露面控制还是后果控制，对多层防护组合联动的逻辑理解仍不够透彻。

3.本次作业中最需要教师复核的段落：SQL 注入因果链与防护对应关系表部分，各流程环节和防护措施的匹配关联、层级划分内容，需要老师检查逻辑是否严谨、对应关系是否准确。

4.如果重做一次，我会优先修改的部分：优化防护清单各项措施的表述，进一步精简语言、强化严谨性，同时完善因果链的逻辑衔接，让攻击流程和防护手段的对应关系更加直观清晰。

收口项 3：AI 输出审核与人工复核记录

1.本次作业中，SQL 注入防护清单撰写、防护措施层级划分、SQL 注入因果链防护关系表内容，均使用 AI 辅助完成内容框架与文字撰写。

2.人工复核后自主保留的判断：区分根因控制、暴露面控制、后果范围控制三类防护层级，各安全措施对应的作用环节、核心解决问题，以及作业整体核心知识点与答题逻辑。

3.复核过程中发现部分 AI 表述冗余、分类模糊，存在防护措施层级混淆、语句繁琐不够严谨的问题，已手动精简文字、修正层级对应关系，删除多余拓展内容，统一答题规范。

4.最终对照课程知识点、SQL 注入攻击流程与防护原理，逐句核对每一项措施的作用、风险环节、防护类型，确保每条结论匹配对应原理依据，实现知识点、作答结论、原理证据一一对应，保证内容准确合规。